

CORSO NODE.JS + EXPRESS + JWT

Validazione, bcrypt e registrazione

Dispensa del corso

Giorno 2 — Parte 2

DOCENTE

Massimo Petrolati

Product Manager · Namirial SpA

Edizione 2026 · Documento di studio per studenti livello base

Indice

Cosa troverai in questa dispensa

1	Validazione input con <code>express-validator</code>	7
1.1	Perché non ci si fida MAI del client	7
1.2	<code>express-validator</code> : cos'è	7
1.3	Sintassi base	7
1.4	Validatori built-in più usati	8
1.5	Custom validators (password complessa)	9
1.6	Costruire <code>authValidators.js</code>	9
1.7	Il middleware <code>validate</code>	10
1.8	Mettere tutto insieme: la catena di middleware	11
2	Configurazione e segreti con <code>dotenv</code>	14
2.1	Cos'è una variabile d'ambiente	14
2.2	Perché i segreti NON vanno nel codice	14
2.3	<code>dotenv</code> in pratica	14
2.4	<code>.gitignore</code> : la regola d'oro	15
2.5	<code>config/env.js</code> : centralizzare con fail-fast	16
2.6	Development vs Production	17
3	Hashing password con <code>bcrypt</code>	19
3.1	Perché NON si salva mai la password in chiaro	19
3.2	Perché non basta MD5 o SHA1	19
3.3	<code>bcrypt</code> : salt + work factor	19
3.4	<code>bcrypt</code> vs <code>bcryptjs</code>	20
3.5	Le due funzioni che useremo	20
3.6	Il wrapper <code>hashPassword.js</code>	21
3.7	Esperimento: hash diverso ma compare sempre true	21
4	Service layer: <code>authService.register</code>	24
4.1	Cos'è il service layer	24
4.2	<code>authService.register</code> passo passo	24
4.3	Difesa in profondità: il DB ti copre comunque	26
4.4	Testabilità	26
5	Controller, route e test end-to-end	28

5.1	<code>authController.register</code>	28
5.2	<code>authRoutes.js</code> : la catena completa	29
5.3	Wire-up in <code>app.js</code>	29
5.4	Test end-to-end con Postman	30
5.5	Apri il file <code>.db</code> e vedi cosa hai scritto	32
5.6	Esercizio extra: campo <code>role</code> opzionale	32

Introduzione

Il primo endpoint serio del progetto

Bentornato al pomeriggio del Giorno 2. Stamattina abbiamo costruito le fondamenta della nostra applicazione: error handler centralizzato, SQLite, repository pattern. Ma manca ancora il pezzo che fa la differenza tra “un’app scolastica” e “un’app pronta da usare”: la **qualità degli input** e la **sicurezza delle password**.

In queste due ore costruiamo il primo endpoint *serio* dell'API: `POST /api/auth/register`. Riceve email, password e nome, valida tutto, hasha la password con un algoritmo serio, scrive l'utente su SQLite e risponde con 201 Created. Sembra banale, ma in mezzo ci sono **quattro concetti professionali** che ti porterai dietro per tutta la carriera.

I quattro concetti di oggi

- **Validazione input sul server:** non ci si fida MAI del client. Mai. Useremo `express-validator` per dichiarare in modo pulito che cosa accetta ogni endpoint, e un middleware unico per restituire errori uniformi.
- **Configurazione tramite variabili d'ambiente:** i segreti (chiavi JWT, password DB) non vivono nel codice. Li tireremo fuori in un file `.env` non committato, leggendoli con `dotenv`, e centralizzandoli in `config/env.js` con fail-fast all'avvio.
- **Hashing password con `bcrypt`:** capirai *perché* MD5 e SHA1 non bastano, cosa sono salt e work factor, e perché `bcrypt.hash("ciao")` produce ogni volta un hash diverso ma `compare` torna sempre `true`.
- **Service layer:** dove vive la logica di business. Sopra al repository, sotto al controller. È il pezzo che orchestra tutto: “cerca duplicati, hasha, salva, ritorna utente pulito”.

A fine giornata avremo

Un endpoint `POST /api/auth/register` che:

- Riceve `{ email, password, name }` in JSON.
- Valida ogni campo con regole esplicite (`email` ben formata, `password` di almeno 8 caratteri con maiuscola e numero, `name` opzionale).
- Risponde **400** con elenco errori se la validazione fallisce.
- Risponde **409** se l'email è già usata.

- Hasha la password con `bcrypt` prima di salvarla.
- Salva su SQLite via `userRepository.create`.
- Risponde **201 Created** con l'utente (senza `password_hash`, ovviamente).

Come è strutturata

La dispensa segue i cinque blocchi della lezione:

- Validazione input con `express-validator`: regole, custom validators, middleware `validate`.
- Configurazione e segreti: `dotenv`, `.env`, `config/env.js` con fail-fast.
- Hashing password con `bcrypt`: teoria + utility `hashPassword` / `comparePassword`.
- Service layer: `authService.register` con orchestrazione completa.
- Controller, route e test end-to-end con Postman.

I box che incontrerai

- **CODICE:** riferimento al mini-progetto pronto nella cartella `codice-esempi/giorno-2-parte-2/`.
- **Suggerimento:** consiglio pratico per evitare problemi tipici.
- **Attenzione:** trappola comune o cosa NON fare.

Come imparare davvero

Oggi è un caso unico nel corso: a fine giornata avrai un endpoint **funzionante e testabile con Postman** che potresti già mostrare a un colloquio di lavoro. Non passare avanti finché non hai visto il tuo `201 Created` apparire davanti agli occhi: è il momento di consolidamento più potente del corso.

Dove trovi il codice

Tutti gli esempi della dispensa sono pronti nel **repository GitHub del corso**:

github.com/mpetrolati/2026-corso-backend-nodejs-express-its

Se non l'hai ancora clonato:

```
$ git clone https://github.com/mpetrolati/2026-corso-backend-nodejs-express-its.git
$ cd 2026-corso-backend-nodejs-express-its
```

I file di questa dispensa stanno nella cartella `codice-esempi/giorno-2-parte-2/`.

CAPITOLO 01

Validazione input con express-validator

Non fidarsi mai del client: regole, custom validators, middleware uniforme

1. Validazione input con `express-validator`

1.1 Perché non ci si fida MAI del client

È la prima regola della sicurezza web, ed è quella che a livello junior viene spesso dimenticata: **tutto quello che arriva da fuori va validato dal tuo server, sempre.**

Non importa se il tuo frontend già fa la validazione lato browser: il client può essere:

- Un utente normale, ma con il frontend rotto da un bug.
- Un utente che ha disabilitato JavaScript.
- Un client diverso dal tuo frontend: Postman, una app mobile, un altro server.
- Un attaccante che fa richieste a mano con `curl` per testare cosa accetti.

La **validazione lato server** è l'unica vera difesa. Quella lato client è solo un favore all'utente normale (gli dai feedback prima senza mandare la richiesta a vuoto).

Cosa significa “validare”

Vuol dire rispondere a tre domande per ogni campo del body: (1) è **presente** quando deve esserlo? (2) è del **tipo giusto** (stringa, numero, array)? (3) è dentro al **range/formato** che ti aspetti (lunghezza, regex, valori ammessi)? Se una qualunque delle risposte è no, l'API risponde 400 Bad Request prima di toccare il DB o la logica.

1.2 `express-validator`: cos'è

`express-validator` è la libreria standard per la validazione in Express. È un wrapper sopra `validator.js` (una libreria con centinaia di validatori già pronti: email, URL, UUID, JWT, IBAN, eccetera) con un'API a catena di middleware.

```
$ npm install express-validator
```

1.3 Sintassi base

Si dichiarano le regole come una **lista di middleware**. Ogni regola opera su un campo del body, query o params e dichiara cosa accetta.

```
import { body } from 'express-validator';

const registerValidator = [
  body('email')
    .isEmail().withMessage('Email non valida')
    .normalizeEmail(),

  body('password')
    .isString().withMessage('Password obbligatoria')
    .isLength({ min: 8 }).withMessage('Password troppo corta'),

  body('name')
    .optional()
    .isString()
    .isLength({ min: 2, max: 50 })
];
```

Cose interessanti:

- `body('email')` prende il campo `email` dal body. Esistono anche `query(...)`, `param(...)`, `header(...)` per gli altri posti.
- I metodi sono a catena: `.isEmail().withMessage(...).normalizeEmail()`. Ognuno applica una regola o una trasformazione.
- `.optional()` dice che il campo può mancare. Senza, il campo è obbligatorio.

1.4 Validatori built-in più usati

La lista è lunga, ma quelli che useremo in pratica:

- `.isEmail()`: email ben formata
- `.isString()` `.isInt()` `.isFloat()` `.isBoolean()`: tipo di base
- `.isLength({ min: N, max: M })`: lunghezza
- `.matches(/regex/)`: regex personalizzata
- `.isIn(['user', 'admin'])`: whitelist di valori ammessi
- `.isUUID()` `.isJWT()` `.isURL()`: formati specifici
- `.optional()`: il campo può mancare
- `.trim()`: toglie spazi a inizio/fine (è un **sanitizer**, modifica il valore)
- `.normalizeEmail()`: normalizza l'email (minuscolo, ecc.)
- `.withMessage('...')`: messaggio di errore se la regola precedente fallisce

1.5 Custom validators (password complessa)

Quando un validator built-in non basta, ne scrivi uno custom con `.custom(fn)`. Esempio: una password che deve contenere almeno una maiuscola e un numero.

```
body('password')
  .isLength({ min: 8 }).withMessage('Password troppo corta')
  .custom((value) => {
    if (!/[A-Z]/.test(value)) {
      throw new Error('Serve almeno una maiuscola');
    }
    if (!/[0-9]/.test(value)) {
      throw new Error('Serve almeno un numero');
    }
    return true; // tutto ok
  });
```

La funzione custom segue una convenzione: lancia `Error` se non va bene, ritorna `true` (o un valore *truthy*) se va bene.

Sanitizer vs Validator

Un **validator** controlla che il valore rispetti una regola, senza modificarlo. Un **sanitizer** modifica il valore (es. `trim` toglie spazi, `normalizeEmail` mette in minuscolo). Express-validator li distingue ma li mischia nella stessa catena: il valore arriva al controller già pulito.

1.6 Costruire `authValidators.js`

Mettiamo in un solo file le regole per gli endpoint di autenticazione. Useremo solo `register` oggi; `login` arriverà domani.

```
// src/validators/authValidators.js
import { body } from 'express-validator';

export const registerValidator = [
  body('email')
    .isEmail().withMessage('Email non valida')
    .normalizeEmail(),

  body('password')
    .isString().withMessage('Password obbligatoria')
    .isLength({ min: 8 }).withMessage('Password di almeno 8 caratteri')
    .custom((value) => {
      if (!/[A-Z]/.test(value)) throw new Error('Serve almeno una maiuscola');
      if (!/[0-9]/.test(value)) throw new Error('Serve almeno un numero');
      return true;
    }),

  body('name')
    .optional()
    .isString()
    .trim()
    .isLength({ min: 2, max: 50 }).withMessage('Nome tra 2 e 50 caratteri'),

  body('role')
    .optional()
    .isIn(['user', 'admin']).withMessage('Role deve essere user o admin')
];

export const loginValidator = [
  body('email').isEmail().normalizeEmail(),
  body('password').isString().notEmpty()
];
```

1.7 Il middleware validate

La catena di validatori **popola** `req` con i risultati ma non risponde al client se ci sono errori. Quello lo fa un middleware separato che leggiamo subito dopo i validatori. Lo scriviamo una volta e lo riusiamo per ogni endpoint validato.

```
// src/middlewares/validate.js
import { validationResult } from 'express-validator';

export function validate(req, res, next) {
  const errors = validationResult(req);
  if (errors.isEmpty()) return next();

  return res.status(400).json({
    errore: 'Validazione fallita',
    dettagli: errors.array().map(e => ({
      campo: e.path,
      messaggio: e.msg
    }))
  });
}
```

1.8 Mettere tutto insieme: la catena di middleware

Nella route, prima i validatori, poi il middleware `validate`, poi il controller. Express attraversa la pipeline come una catena: se `validate` risponde con 400, il controller non viene mai chiamato.

```
// src/routes/authRoutes.js
import { Router } from 'express';

import { registerValidator } from '../validators/authValidators.js';
import { validate } from '../middlewares/validate.js';
import * as authController from '../controllers/authController.js';

const router = Router();

router.post('/register',
  registerValidator, // catena di validatori
  validate,          // se ci sono errori, risponde 400
  authController.register
);

export default router;
```

Pulizia del controller

Hai notato? Il controller `authController.register` non ha più *nessun if di validazione*. È pulito, fa solo logica di business. Ogni controllo sull'input vive nei validatori. Questa separazione è oro.

CODICE: giorno-2-parte-2/01-express-validator/

CAPITOLO 02

Configurazione e segreti con dotenv

File .env, fail-fast all'avvio, separazione dev/prod

2. Configurazione e segreti con dotenv

2.1 Cos'è una variabile d'ambiente

Una **variabile d'ambiente** è un valore che il sistema operativo mette a disposizione dei processi che lancia. In Node ci accedi tramite l'oggetto globale `process.env`:

```
console.log(process.env.HOME);      // /home/mario
console.log(process.env.PATH);      // /usr/bin:/usr/local/bin:...
console.log(process.env.NODE_ENV);  // se settata, "development" / "production"
```

In produzione, le variabili d'ambiente sono il modo standard per passare configurazione a un processo senza scriverla nel codice: porta, URL del database, chiavi API, password del DB, secret JWT.

2.2 Perché i segreti NON vanno nel codice

Questo merita uno spazio dedicato perché è un errore tipico da junior. Mai mettere chiavi, password, secret JWT direttamente nel codice sorgente, neanche temporaneamente per “ci penso dopo”. Quattro motivi pratici:

- **Git è per sempre.** Se committi un secret per sbaglio, anche dopo averlo rimosso resta nello storico Git. Su GitHub pubblico, in pochi minuti dei bot lo scansiano e lo usano per fare danni.
- **Ambienti diversi hanno valori diversi.** Il DB di sviluppo non è il DB di produzione. Hardcodare ti obbliga a modificare il codice per cambiare ambiente.
- **Rotation:** i secret vanno cambiati periodicamente. Cambiarli nel codice significa fare deploy ogni volta. Cambiarli come variabili d'ambiente significa solo riavviare il processo.
- **Sicurezza in azienda:** chi può leggere il codice non deve necessariamente conoscere i secret di produzione. Separare il codice (Git) dai segreti (variabili d'ambiente) implementa questa separazione.

2.3 dotenv in pratica

`dotenv` è la libreria standard per caricare un file `.env` dentro `process.env` durante lo sviluppo. In produzione le variabili arrivano dal sistema (Docker, Kubernetes, Render, Railway, eccetera), non da `.env`.

```
$ npm install dotenv
```

Crea il file `.env` alla radice del progetto:

```
# Server
PORT=3000
NODE_ENV=development

# Database
DB_PATH=./data/app.db

# JWT (li useremo nel Giorno 3)
JWT_SECRET=cambia-questa-stringa-in-produzione
JWT_EXPIRES_IN=1h

# Bcrypt
BCRYPT_ROUNDS=10
```

E un `.env.example` con le stesse chiavi ma valori *finti*, da committare:

```
PORT=3000
NODE_ENV=development
DB_PATH=./data/app.db
JWT_SECRET=metti-qui-una-stringa-lunga-e-casuale
JWT_EXPIRES_IN=1h
BCRYPT_ROUNDS=10
```

Il `.env.example` serve come “contratto”: chi clona il progetto sa quali variabili deve definire, senza vedere i tuoi valori reali.

2.4 .gitignore: la regola d'oro

Aggiungi `.env` al `.gitignore` **prima** di committare qualunque cosa.

```
node_modules/
.env
.env.local
*.log
.DS_Store
data/
```

Se hai già committato .env per sbaglio

Rimuoverlo con `git rm --cached .env` non basta: resta nello storico. Devi rigenerare TUTTI i secret committati (cambia la password del DB, ruota la chiave JWT, eccetera) e poi rimuoverlo dallo storico con `git filter-repo` (avanzato). Lezione: il momento giusto per scrivere il `gitignore` è prima del primo commit.

2.5 config/env.js: centralizzare con fail-fast

Invece di disseminare `process.env.JWT_SECRET` per tutto il codice, lo centralizziamo in un solo file: `src/config/env.js`. Questo file:

- Carica `dotenv` all'avvio (una volta sola).
- Legge tutte le variabili che ci servono.
- Se ne manca una obbligatoria, fa crashare l'app subito (**fail-fast**). Meglio crashare al boot con un messaggio chiaro, che andare in produzione e ricevere errori misteriosi alla prima richiesta.

```
// src/config/env.js
import 'dotenv/config';

function required(name) {
  const value = process.env[name];
  if (!value) {
    console.error(`[config] Variabile d'ambiente mancante: ${name}`);
    process.exit(1);
  }
  return value;
}

export const env = {
  port:      parseInt(process.env.PORT || '3000', 10),
  nodeEnv:   process.env.NODE_ENV || 'development',
  dbPath:    process.env.DB_PATH || './data/app.db',
  jwtSecret: required('JWT_SECRET'),
  jwtExpiresIn: process.env.JWT_EXPIRES_IN || '1h',
  bcryptRounds: parseInt(process.env.BCRYPT_ROUNDS || '10', 10)
};
```

Da ora in poi, qualunque file che vuole leggere configurazione importa da qui:


```
import { env } from '../config/env.js';

console.log('Server in ascolto sulla porta', env.port);
```

2.6 Development vs Production

La variabile `env` è una convenzione (non un'imposizione) di Node per distinguere gli ambienti. Express e tantissime librerie cambiano comportamento in base a essa:

- In `development`: messaggi di errore dettagliati, stack trace al client, hot reload.
- In `production`: messaggi di errore minimali, ottimizzazioni accese, caching aggressivo.

In produzione, il file `.env` **non esiste**. Le variabili arrivano da:

- Variabili settate nel sistema operativo (`export JWT_SECRET=...`).
- Variabili definite in Docker (`ENV` o `--env-file`).
- Variabili passate dalla piattaforma di deploy (Render, Railway, Heroku, Fly.io).
- Servizi di gestione segreti dedicati (AWS Secrets Manager, Vault).

Una buona pratica

In `server.js`, all'avvio, logga le variabili di configurazione *non sensibili* (porta, `NODE_ENV`, percorso DB). Aiuta moltissimo in fase di debug per capire *in quale ambiente* sta girando l'app. Mai loggare i secret, ovviamente.

CODICE: `giorno-2-parte-2/02-dotenv-config/`

CAPITOLO 03

Hashing password con bcrypt

Salt, work factor, perché MAI in chiaro

3. Hashing password con bcrypt

3.1 Perché NON si salva mai la password in chiaro

Sembra ovvio detto così, ma vediamo *perché*, con tre scenari realistici.

- **Il database viene compromesso** (attacco, furto di backup, dipendente disonesto). Se le password sono in chiaro, l'attaccante ha tutto. E siccome le persone riusano le stesse password ovunque, può loggarsi sui loro account di posta, banca, social.
- **Tu stesso non puoi vederle.** Una password è un segreto tra l'utente e il sistema. Non c'è motivo per cui l'admin del DB debba poterla leggere. Se la vede, ha una responsabilità in più che è meglio non avere.
- **Privacy by design:** hashare le password è un requisito legale (GDPR) e una buona pratica universale. Non è opzionale.

3.2 Perché non basta MD5 o SHA1

Storicamente, prima si usavano funzioni hash crittografiche generiche: MD5, SHA1, SHA256. Producono un hash dalla password, e il DB salva l'hash invece della password. Ma c'è un problema: queste funzioni sono **deterministiche e veloci**.

- **Deterministiche:** `md5("ciao")` dà sempre lo stesso risultato. Quindi se due utenti hanno la stessa password, il DB ha lo stesso hash: l'attaccante lo vede.
- **Veloci:** una GPU moderna può calcolare miliardi di MD5 al secondo. Se la password è una di quelle "classiche" (`password`, `123456`, `ciao`), un attaccante la trova in millisecondi con un **rainbow table**: una grande tabella precomputata di password → hash.

Le funzioni hash generiche sono pensate per essere veloci: utile per verificare l'integrità di un file (calcoli e confronti subito), **catastrofico** per le password.

3.3 bcrypt: salt + work factor

bcrypt è un algoritmo di hashing pensato apposta per le password. Risolve i due problemi di MD5/SHA1 con due idee:

- **Salt:** ogni hash include una stringa casuale chiamata salt. Anche se due utenti hanno la stessa password, gli hash sono diversi perché il salt è diverso. Le rainbow table non funzionano più.
- **Work factor (rounds):** **bcrypt** è apposta lento. Il parametro `rounds` dice quante iterazioni fare. Con `rounds=10` servono circa 100ms per hashare una password. Per un utente legittimo non si nota; per un attaccante che vuole brute-forzare milioni di hash diventa proibitivo.

Un hash bcrypt sembra così'

```
$2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy
```

La struttura è: `$algoritmo$rounds$salt+hash`. Tutto in un solo campo TEXT, niente schema speciale nel DB. Salvi questa stringa così com'è.

3.4 bcrypt vs bcryptjs

Esistono due implementazioni:

- `bcrypt` (la libreria nativa, scritta in C++): più veloce, ma va compilata su Windows.
- `bcryptjs` (puro JavaScript): un po' più lenta, ma non richiede compilazione. Piano B se hai problemi su Windows.

L'API è identica al 95%. Useremo `bcrypt`, ma se incontri errori di build durante `npm install` su Windows, puoi sostituirlo con `bcryptjs` cambiando solo l'import.

```
$ npm install bcrypt
# (piano B su Windows: npm install bcryptjs)
```

3.5 Le due funzioni che useremo

`bcrypt` ha decine di API, ma ne useremo solo due, sempre nella forma `async/await`:

- `bcrypt.hash(plain, rounds)`: hash la password. Ritorna una Promise di stringa.
- `bcrypt.compare(plain, hash)`: confronta una password con un hash. Ritorna una Promise di boolean.

```
import bcrypt from 'bcrypt';

const ROUNDS = 10;

const hash = await bcrypt.hash('passwordSegreta123', ROUNDS);
// $2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy

const ok1 = await bcrypt.compare('passwordSegreta123', hash); // true
const ok2 = await bcrypt.compare('passwordSbagliata', hash);  // false
```

3.6 Il wrapper hashPassword.js

Per non importare `bcrypt` sparso ovunque, lo wrappiamo in un piccolo modulo `utils/`. Così, il giorno che decidiamo di passare a `bcryptjs` o `argon2`, cambiamo un solo file.

```
// src/utils/hashPassword.js
import bcrypt from 'bcrypt';
import { env } from '../config/env.js';

export async function hashPassword(plain) {
  return bcrypt.hash(plain, env.bcryptRounds);
}

export async function comparePassword(plain, hash) {
  return bcrypt.compare(plain, hash);
}
```

I rounds sono presi da `env.bcryptRounds` (configurabile via `.env`). Non hardcodare nemmeno questo: in test si abbassa a 4 per velocità, in produzione si alza a 12.

3.7 Esperimento: hash diverso ma compare sempre true

Per fissare visivamente l'idea del salt: scriviamo uno script che hash a la stessa password tre volte e mostra che gli hash sono diversi, ma il `compare` ritorna sempre `true`.

```
// playground.js
import bcrypt from 'bcrypt';

const plain = 'ciao';

const h1 = await bcrypt.hash(plain, 10);
const h2 = await bcrypt.hash(plain, 10);
const h3 = await bcrypt.hash(plain, 10);

console.log(h1);
console.log(h2);
console.log(h3);
// tre stringhe DIVERSE (grazie al salt)

console.log(await bcrypt.compare(plain, h1)); // true
console.log(await bcrypt.compare(plain, h2)); // true
console.log(await bcrypt.compare(plain, h3)); // true
```

Confronto = bcrypt.compare, NON ===

È un errore tipico: tentare di confrontare hash con `===` o `equals`. **Non funziona**, perché ogni hash è diverso anche per la stessa password (salt). Devi usare sempre `bcrypt.compare(plain, hash)`: la libreria sa estrarre il salt dall'hash e ricalcolare.

CODICE: `giorno-2-parte-2/03-bcrypt-playground/`

CAPITOLO 04

Service layer:

`authService.register`

Dove vive la logica di business

4. Service layer: `authService.register`

4.1 Cos'è il service layer

Sopra al repository (che parla col DB), sotto al controller (che parla con Express), c'è il **service layer**: dove vive la **logica di business** dell'applicazione.

- Il **repository** è “meccanico”: scrive ciò che gli viene chiesto, legge ciò che gli viene chiesto. Non valida, non hash, non controlla duplicati.
- Il **controller** è “protocollare”: estrae i dati da `req`, chiama il service, scrive in `res`. Non sa di logica.
- Il **service** è “cervello”: orchestra. Decide *cosa* fare. Cerca duplicati, hash, salva, ritorna.

Vantaggio principale

Il service **non sa nulla di Express**. Non vede `req` né `res`. Si può testare in isolamento: gli passi un oggetto `{ email, password, name }`, ti ritorna un utente o lancia un'eccezione. È testabile come una pura funzione.

4.2 `authService.register` passo passo

La funzione che orchestra la registrazione:


```
// src/services/authService.js
import * as userRepository from '../repositories/userRepository.js';
import { hashPassword } from '../utils/hashPassword.js';
import { AppError } from '../utils/AppError.js';

export async function register({ email, password, name = null, role = 'user' }) {
  // 1. C'è già un utente con questa email?
  const existing = userRepository.findByEmail(email);
  if (existing) {
    throw new AppError('Email già registrata', 409);
  }

  // 2. Hash della password
  const passwordHash = await hashPassword(password);

  // 3. Scrivo l'utente nel DB
  const user = userRepository.create({
    email,
    passwordHash,
    name,
    role
  });

  // 4. Ritorno l'utente pulito (senza password_hash)
  return toSafeUser(user);
}

function toSafeUser(user) {
  const { password_hash, ...safe } = user;
  return safe;
}
```

Da fissare:

- **Check duplicato *prima* di hashare:** l'hashing è lento (centinaia di ms con `rounds=10`). Se sappiamo già che l'email è duplicata, evitiamo di consumare CPU.
- **409 Conflict** è lo status code REST corretto per “risorsa già esistente”. Non 400, non 500.
- Il service **lancia** l'errore (non chiama `next(err)`): è il controller, sopra, che cattura con `asyncHandler`. Il service non conosce Express.
- **toSafeUser**: stessa funzione che avevamo nel CRUD del Giorno 2 Parte 1. La spostiamo qui dentro al service. Da ora in poi tutte le route che chiamano `authService` ricevono utenti già puliti, senza dover ricordarsi di togliere `password_hash`.
- **Tutto è `async`**: usiamo `await` per l'hash, che è asincrono. Il service è una funzione `async`, il controller la chiamerà con `await`.

4.3 Difesa in profondità: il DB ti copre comunque

Ricordi il vincolo `UNIQUE` sull'email che abbiamo messo nello schema della tabella `users`? È la nostra **ultima linea di difesa**.

Anche se il check `findByEmail` fallisse (per una race condition, un bug, qualunque cosa), il DB rifiuterebbe l'inserimento con un errore `SQLITE_CONSTRAINT_UNIQUE`. Il nostro `errorHandler` (vedi Giorno 2 Parte 1) mappa quell'errore a 409, quindi il client riceve comunque la risposta giusta.

Questo principio si chiama **defense in depth**: avere più strati di difesa, così se uno fallisce, gli altri ti coprono.

4.4 Testabilità

Il service è una funzione pura (modulo il side effect di scrivere su DB). Per testarlo veramente in isolamento basta sostituire `userRepository` con un mock. Non lo facciamo nel corso, ma per dare un'idea:

```
// pseudo-test (non eseguibile così, serve un framework tipo jest)
import { register } from '../src/services/authService.js';

test('rifiuta email duplicata', async () => {
  // mock del repository: findByEmail ritorna sempre un utente
  jest.mock('../src/repositories/userRepository.js', () => ({
    findByEmail: () => ({ id: 1, email: 'gia@example.com' })
  }));

  await expect(register({
    email: 'gia@example.com',
    password: 'Password123'
  })).rejects.toThrow('Email gia registrata');
});
```

Mostralo solo per dare l'idea: la testabilità di questo livello è enormemente più alta di un controller che fa tutto.

CODICE: `giorno-2-parte-2/04-auth-service-register/`

CAPITOLO 05

Controller, route e test end-to-end

La pipeline completa di `POST /api/auth/register`

5. Controller, route e test end-to-end

5.1 authController.register

Il controller estrae i dati da `req.body`, chiama il service, scrive la risposta. È sottile per costruzione: tutta la logica è nel service, tutta la validazione è nei validatori, qui c'è solo l'*orchestrazione HTTP*.

```
// src/controllers/authController.js
import * as authService from '../services/authService.js';
import { asyncHandler } from '../utils/asyncHandler.js';

export const register = asyncHandler(async (req, res) => {
  const user = await authService.register(req.body);
  res.status(201).json(user);
});
```

Tre righe utili, sei col boilerplate. Notare:

- `asyncHandler` cattura le promise rifiutate. Se `authService.register` lancia un `AppError`, l'errore arriva all'`errorHandler` senza `try/catch` esplicito.
- Status code `201 Created`, non `200`: stiamo creando una risorsa nuova.
- Il body di risposta è l'utente *pulito* (già senza `password_hash`): il service ha già fatto il lavoro.

5.2 authRoutes.js: la catena completa

```
// src/routes/authRoutes.js
import { Router } from 'express';

import { registerValidator } from '../validators/authValidators.js';
import { validate } from '../middlewares/validate.js';
import * as authController from '../controllers/authController.js';

const router = Router();

router.post('/register',
  registerValidator,    // 1) regole di validazione
  validate,            // 2) se errori, risponde 400
  authController.register // 3) altrimenti, chiama il controller
);

export default router;
```

Quattro middleware in sequenza, una sola route. La pipeline è esplicita e leggibile: **validatori** → **validate** → **controller** → **service** → **repository** → **DB**.

5.3 Wire-up in app.js

L'unico cambiamento nel file principale è montare il router auth sotto `/api/auth`:

```
// src/app.js
import express from 'express';
import morgan from 'morgan';

import authRoutes from './routes/authRoutes.js';
import userRoutes from './routes/userRoutes.js';
import { errorHandler } from './middlewares/errorHandler.js';

const app = express();

app.use(express.json());
app.use(morgan('dev'));

app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);

app.use((req, res) => res.status(404).json({ errore: 'Route non trovata' }));
app.use(errorHandler);

export default app;
```

5.4 Test end-to-end con Postman

Lancia `npm run dev` e prova questi quattro scenari con Postman o REST Client.

1. Happy path

```
POST http://localhost:3000/api/auth/register
Content-Type: application/json

{
  "email": "mario@example.com",
  "password": "Password123",
  "name": "Mario Rossi"
}
```

Risposta attesa: 201 Created

```
{
  "id": 1,
  "email": "mario@example.com",
  "name": "Mario Rossi",
  "role": "user",
  "created_at": "2026-05-16 10:23:45",
  "updated_at": "2026-05-16 10:23:45"
}
```

Notare che `password_hash` **non c'è** nella risposta: il service lo ha tolto.

2. Validazione fallita: email mal formata

```
POST http://localhost:3000/api/auth/register
Content-Type: application/json

{
  "email": "non-e-una-email",
  "password": "Password123"
}
```

Risposta attesa: 400 Bad Request

```
{
  "errore": "Validazione fallita",
  "dettagli": [
    { "campo": "email", "messaggio": "Email non valida" }
  ]
}
```

3. Validazione fallita: password troppo semplice

```
POST http://localhost:3000/api/auth/register
Content-Type: application/json

{
  "email": "anna@example.com",
  "password": "abc"
}
```

Risposta attesa: 400 Bad Request con più errori in elenco:

```
{
  "errore": "Validazione fallita",
  "dettagli": [
    { "campo": "password", "messaggio": "Password di almeno 8 caratteri" }
  ]
}
```

4. Email già registrata

Manda di nuovo la stessa richiesta del punto 1 (stessa email):

Risposta attesa: 409 Conflict

```
{
  "errore": "Email già registrata"
}
```

5.5 Apri il file `.db` e vedi cosa hai scritto

Mentre il server gira, apri `data/app.db` con DB Browser for SQLite. Vai sulla tabella `users` e guarda la riga di Mario. Vedrai:

- `email`: `mario@example.com`.
- `password_hash`: una stringa lunga e illeggibile tipo `$2b$10$N9qo8uL0...`.
- `role`: `user`.
- `created_at` e `updated_at`: il momento della registrazione.

La password “Password123” non c’è da nessuna parte, e non c’è modo di ricavarla da quel hash. È il punto in cui l’autenticazione smette di essere “didattica” e diventa davvero sicura.

5.6 Esercizio extra: campo `role` opzionale

Se hai tempo, prova ad aggiungere il campo `role` opzionale al body di registrazione, già validato come `oneOf(['user', 'admin'])` (lo abbiamo nei `authValidators`!). Provalo con:


```
POST http://localhost:3000/api/auth/register
Content-Type: application/json
```

```
{
  "email": "boss@example.com",
  "password": "AdminPwd123",
  "role": "admin"
}
```

E poi prova un valore non valido:

```
{
  "email": "x@example.com",
  "password": "AdminPwd123",
  "role": "superuser"
}
```

Dovresti ricevere **400 Bad Request** con il messaggio del validator. Ottimo terreno per la discussione su RBAC del Giorno 3 (anche se l'implementazione non la facciamo nel corso).

CODICE: `giorno-2-parte-2/05-register-endpoint-completo/`

Appendice

Cosa porti a casa e dove andiamo domani

Cosa ti porti a casa da questa lezione

- La regola d'oro: **non ci si fida MAI del client**. Validazione lato server, sempre.
- `express-validator`: validatori a catena, custom validators, middleware `validate` per risposte 400 uniformi.
- `dotenv` per la configurazione, `.env` (mai committato) + `.env.example` (committato), `config/env.js` con **fail-fast**.
- **Perché** le password vanno hashate, perché MD5/SHA1 non bastano, cosa sono **salt** e **work factor**.
- Il wrapper `hashPassword/comparePassword` su `bcrypt`, configurabile via `env.bcryptRounds`.
- Il **service layer** come “cervello” dell'applicazione, separato da Express.
- Un endpoint `POST /api/auth/register` **completo e funzionante**: validazione → check duplicato → hash → scrittura DB → 201 con utente pulito.

Cartella codice di riferimento

Tutti gli esempi sono pronti nella cartella `codice-esempi/giorno-2-parte-2/` del repository GitHub del corso:

github.com/mpetrolati/2026-corso-backend-nodejs-express-its

Se non l'hai ancora fatto, clonalo (o scaricalo come ZIP):

```
$ git clone https://github.com/mpetrolati/2026-corso-backend-nodejs-express-its.git
$ cd 2026-corso-backend-nodejs-express-its/codice-esempi/giorno-2-parte-2
```

Ogni sottocartella è un mini-progetto autonomo con `package.json`, `.env.example` e `README.md`:

- `01-express-validator/` validatori a catena, custom validator, middleware `validate`
- `02-dotenv-config/` `.env`, `config/env.js` con fail-fast
- `03-bcrypt-playground/` hash diversi ma compare sempre true
- `04-auth-service-register/` il service `register` testato in isolamento

- `05-register-endpoint-completo/` l'endpoint completo, pronto da testare con Postman

Cosa faremo domani (Giorno 3)

Domani completiamo l'autenticazione: **login** con JWT e **route protette**.

- **Teoria JWT**: header, payload, signature. Perché funziona, cosa non bisogna metterci dentro.
- **POST /api/auth/login**: verifica credenziali con `comparePassword`, genera JWT con `jsonwebtoken`.
- Middleware `verifyToken`: legge l'header `Authorization: Bearer ...`, verifica firma, popola `req.user`.
- Endpoint protetti: **GET /api/users/me** e **PUT /api/users/me**.
- Test end-to-end con Postman (con variabili d'ambiente di Postman per il token).
- **Wrap-up** su sicurezza, deploy, prossimi passi.

Per approfondire

- **express-validator**: express-validator.github.io, documentazione completa con esempi.
- **validator.js**: github.com/validatorjs/validator.js, la lista di validatori sotto.
- **dotenv**: github.com/motdotla/dotenv, semplice ma con qualche *gotcha*: vale la pena dare un'occhiata.
- **bcrypt**: github.com/kelektiv/node.bcrypt.js, documentazione e benchmark dei rounds.
- **OWASP Password Storage Cheat Sheet**: cerca "OWASP Password Storage Cheat Sheet", la guida ufficiale alle migliori pratiche di storage password.

Un consiglio prima di andare

Stasera, prima di chiudere il PC, prova a **rifare la registrazione da zero** senza guardare la dispensa. Quando ti blocchi, riapri solo il pezzo che ti manca e ricomincia. Se domani arrivi con questo endpoint chiaro in testa, la giornata di JWT sarà una passeggiata. Altrimenti, sarà doppio lavoro.